

Group symbol: **3**

Team: **1**

Project title: Resource Allocation App

Team members *(filled by PM, Team Leader):*

No	Name	Surname	Student ID	Role
1	Carmelo Aidan	Romero	293936	<i>PM, Team Leader</i>
2	Álvaro	Puebla	293867	<i>Team member</i>
3	David	Sosa	293885	<i>Team member</i>
4	Diego	Ceballos	293943	<i>Team member</i>
5	Diego	García	293939	<i>Team member</i>
6	Óscar	López	293871	<i>Team member</i>

4. Construction and tests (S4)

4.1. Implementation

The documentation focuses on the implementation of the software in the form of programming language source codes. You should prepare lists of created artefacts, i.e., source codes, configuration files, files containing program resources, pre-requirements, and result programs. You should supplement the list of artefacts with a description indicating the relationships between them and their nature (type), as well as technical requirements in the scope of their use. The implementation should cover the minimum viable product (MVP) of the system.

The presentation of the application execution is a must for obtaining a positive grade.

4.1.1. Overview

This section documents the implementation of the Resource Management System MVP (Minimum Viable Product). The system is a web-based application built using Spring Boot, following a layered architecture pattern with clear separation of concerns.

4.1.2. Technology Stack

Backend:

- Java 21
- Spring Boot 3.3.3
- Spring Data JPA (Hibernate)
- Spring Security
- Spring Validation
- MySQL 8

Frontend:

- Thymeleaf (server-side templating)
- HTML5/CSS3
- JavaScript (ES6+)

Build & Deployment:

- Gradle 8.x
- Docker & Docker Compose
- Nginx (reverse proxy)
- Let's Encrypt (SSL/TLS)

Testing:

- JUnit 5
- Mockito
- Spring Boot Test
- Spring Security Test

4.1.3. Artifacts List

A. Source Code Artifacts

Entity Layer (src/main/java/.../entity/)

File	Type	Description	Dependencies
User.java	Abstract Entity	Base class for all users; contains common fields (name, email, password, accountStatus)	JPA, Validation, Lombok
Client.java	Entity	Extends User; represents library clients with reservation limits	User.java
Admin.java	Entity	Extends User; represents system administrators	User.java
Resource.java	Abstract Entity	Base class for reservable resources	JPA, Lombok
Book.java	Entity	Extends Resource; represents library books	Resource.java
Room.java	Entity	Extends Resource; represents reservable rooms	Resource.java
Reservation.java	Entity	Represents resource reservations with composite key	Resource, Client, Admin
ReservationId.java	Embeddable	Composite primary key for Reservation (resourceId, endTime)	JPA
AccountStatus.java	Enum	User account states (ACTIVE, SUSPENDED, BLOCKED, PENDING)	-
EndAfterStart.java	Annotation	Custom validation constraint	Validation
EndAfterStartValidator.java	Validator	Validates end time > start time	EndAfterStart

Repository Layer (src/main/java/.../repository/)

File	Type	Description	Dependencies
UserRepository.java	Interface	CRUD + findByEmail	Spring Data JPA
ClientRepository.java	Interface	Client-specific queries	UserRepository
AdminRepository.java	Interface	Admin-specific queries	UserRepository
ResourceRepository.java	Interface	Base resource operations	Spring Data JPA
BookRepository.java	Interface	Book queries (findByIsbn)	ResourceRepository
RoomRepository.java	Interface	Room queries (findByRoomCode)	ResourceRepository
ReservationRepository.java	Interface	Complex reservation queries (overlaps, availability)	Spring Data JPA

Service Layer (src/main/java/.../service/)

File	Type	Description	Dependencies
UserService.java	Service	User management (CRUD, blocking, authentication prep)	Repositories, PasswordEncoder
ResourceService.java	Service	Resource management (CRUD, availability checking)	Repositories
ReservationService.java	Service	Reservation logic (creation, validation, cancellation)	Repositories
QRService.java	Service	QR code decoding using ZXing library	ZXing

Security Layer (src/main/java/.../security/)

File	Type	Description	Dependencies
SecurityConfig.java	Configuration	Spring Security setup (authentication, authorization, CSRF)	Spring Security
JpaUserDetailsService.java	Service	UserDetailsService implementation for authentication	UserRepository, Spring Security
UserValidationFilter.java	Filter	Validates user status on each request (blocks deleted/suspended users)	UserRepository

Controller Layer (src/main/java/.../controller/)

File	Type	Description	Dependencies
HomeController.java	Controller	Main page, login, resource filtering	Services
ResourceController.java	Controller	Resource details, reservations (client-facing)	Services
AdminController.java	Controller	Admin dashboard (user, resource, reservation management)	Services
ScannerController.java	Controller	QR code scanning functionality	QRService, ResourceService
MyErrorController.java	Controller	Custom error page handling	-

Web Forms (src/main/java/.../web/)

File	Type	Description	Dependencies
UserForm.java	DTO	User creation/editing form	Validation
ResourceForm.java	DTO	Resource creation/editing form (polymorphic Book/Room)	Validation
ReservationForm.java	DTO	Client reservation form	Validation, EndAfterStart
AdminReservationForm.java	DTO	Admin reservation form (includes clientId)	Validation, EndAfterStart

Application Entry

File	Type	Description
ResourceManagementApplication.java	Main Class	Spring Boot application entry point

B. Configuration Files

File	Type	Description	Used By
application.properties	Config	Database, JPA, logging, security settings	Spring Boot
build.gradle	Build	Gradle dependencies and build configuration	Gradle
settings.gradle	Build	Project settings	Gradle
docker-compose.yml	Deployment	Container orchestration (nginx, MySQL, backend, certbot)	Docker
Dockerfile	Deployment	Backend container definition	Docker
nginx/conf.d/resourcemanagement.conf	Config	Nginx reverse proxy configuration	Nginx
nginx/start.sh	Script	Dynamic nginx configuration (HTTP/HTTPS switching)	Nginx
mysql/conf/my.cnf	Config	MySQL performance tuning	MySQL
setup-https.sh	Script	Automated SSL certificate acquisition	Certbot

C. Resource Files

Database Initialization

File	Type	Description
src/main/resources/init/data.sql	SQL	Initial data (admin user, client user, sample resources)
src/test/resources/init/data.sql	SQL	Test data for integration tests

Templates (src/main/resources/templates/)

File	Type	Description
layout.html	Template	Master layout with header, footer, navigation
index.html	Template	Main resource browsing page
login.html	Template	Login form
resource.html	Template	Resource detail and reservation page
reservation-edit.html	Template	Client reservation editing
scan.html	Template	QR code scanner interface
error.html	Template	Error page
admin/dashboard.html	Template	Admin overview
admin/resources.html	Template	Resource management table
admin/resource_form.html	Template	Resource creation/editing
admin/user_form.html	Template	User creation
admin/reservation_form.html	Template	Admin reservation creation
admin/reservation_edit.html	Template	Admin reservation editing

Static Resources

Path	Type	Description
static/css/*.css	Stylesheet	11 CSS files for styling (variables, layout, components)
static/js/*.js	JavaScript	5 JS files for client-side behavior
static/img/WUST.png	Image	University logo

Test Resources (src/test/resources/)

File	Type	Description
application.properties	Config	Test database configuration
static/img/validqr.png	Image	Valid QR code for testing
static/img/notvalidqr.png	Image	Invalid QR code for testing

D. Test Artifacts

Unit Tests (13 test classes)

File	Coverage	Test Count
EndAfterStartValidatorTest.java	Entity validation	8 tests
EntityValidationTest.java	Entity constraints	8 tests
QRServiceTest.java	QR code decoding	6 tests
UserServiceTest.java	User management	6 tests
ResourceServiceTest.java	Resource operations	5 tests
ReservationServiceTest.java	Reservation logic	6 tests

Integration Tests

File	Coverage	Test Count
RepositoryIntegrationTest.java	Database layer	8 tests
FullIntegrationTest.java	End-to-end workflows	7 tests

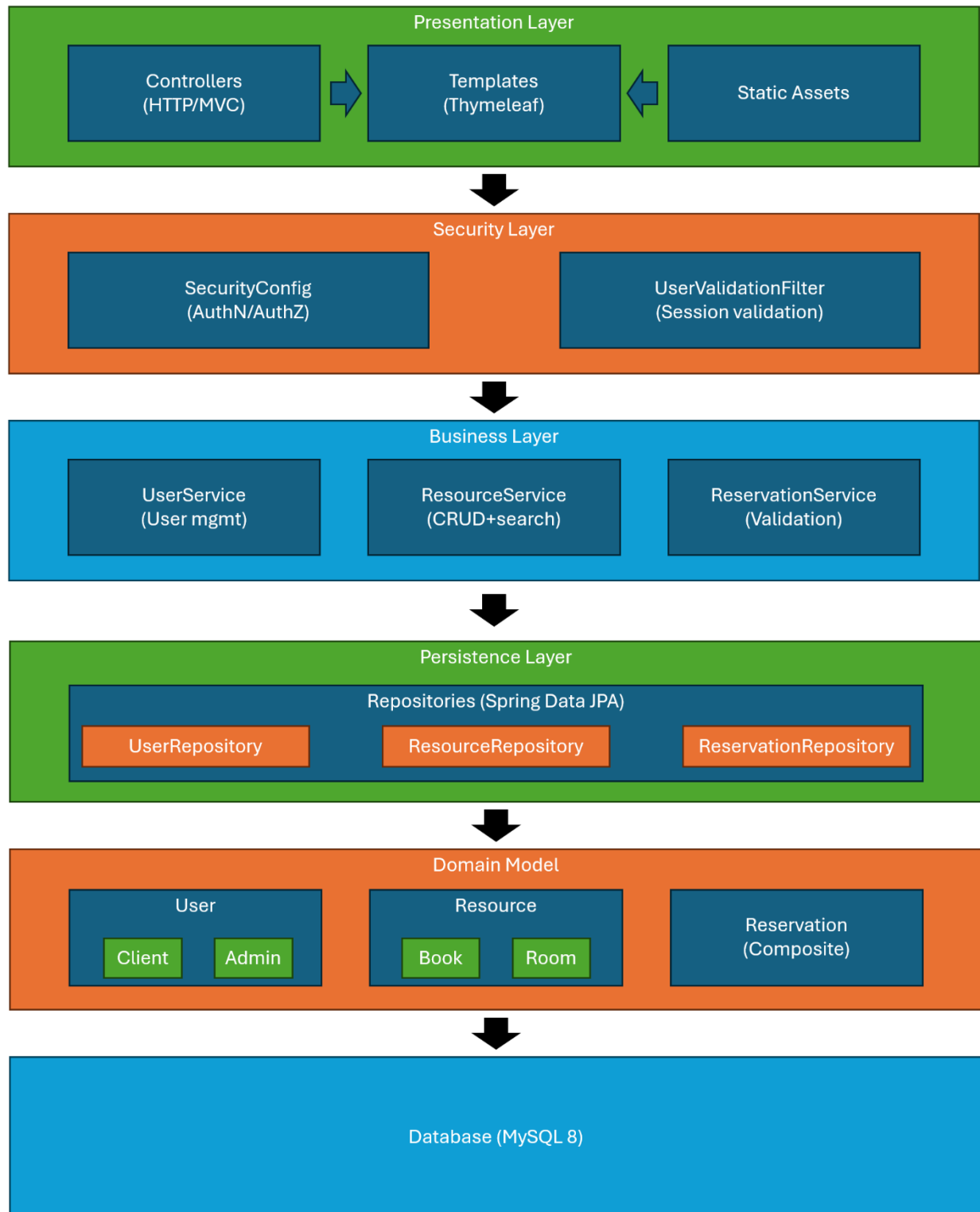
Controller Tests (MockMvc)

File	Coverage	Test Count
AdminControllerTest.java	Admin endpoints	11 tests
HomeControllerTest.java	Public pages	8 tests
ResourceControllerTest.java	Resource operations	5 tests
ScannerControllerTest.java	QR scanning	6 tests

Security Tests

File	Coverage	Test Count
SecurityConfigTest.java	Authentication & authorization	10 tests

4.1.4. Architecture & Relationships



Key Relationships:

1. **Inheritance:** User → Client/Admin, Resource → Book/Room (JOINED strategy)
2. **Composition:** Reservation embeds ReservationId (composite key: resourceId + endTime)
3. **Associations:**
 - Reservation → Resource (ManyToOne, part of PK)
 - Reservation → Client (ManyToOne, required)
 - Reservation → Admin (ManyToOne, optional - set when admin creates reservation)

4.1.5. Technical Requirements

Prerequisites:

- **Java:** JDK 21 or higher
- **Gradle:** 8.x (wrapper included)
- **Docker:** 20.x or higher
- **Docker Compose:** v2.x or higher
- **MySQL:** 8.0 (via Docker or local)
- **Modern Browser:** Chrome/Firefox/Edge (for client UI)

Build & Run:

docker compose up -d --build

```
apueblar@raspberrypi:~/docker/resource-management $ docker compose up -d --build
[+] Building 172.0s (17/17) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 600B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 617B
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [ 1/10] FROM docker.io/library/eclipse-temurin:21-jdk@sha256:e8d57b6c5c73d93212a4384f588309b44bc75fd15b38f78e6a6db1b350dc4ef3
=> => resolve docker.io/library/eclipse-temurin:21-jdk@sha256:e8d57b6c5c73d93212a4384f588309b44bc75fd15b38f78e6a6db1b350dc4ef3
=> [internal] load build context
=> => transferring context: 144.60kB
=> CACHED [ 2/10] WORKDIR /app
=> CACHED [ 3/10] COPY gradlew .
=> CACHED [ 4/10] COPY gradlew.bat .
=> [ 5/10] COPY build.gradle .
=> [ 6/10] COPY settings.gradle .
=> [ 7/10] COPY gradle ./gradle
=> [ 8/10] COPY src ./src
=> [ 9/10] RUN chmod +x ./gradlew
=> [10/10] RUN ./gradlew build -x test
=> exporting to image
=> => exporting layers
=> => writing image sha256:8fa76000baa47abbc7a590c7cb2b4eddf36d5c607f4a9d641a53aaa715c0ef06
=> => naming to docker.io/library/resource-management-backend
=> resolving provenance for metadata file
[+] up 6/6
✔ Image resource-management-backend Built
✔ Network resource-management_resource-net Created
✔ Container resourcemanagement-certbot Created
✔ Container resourcemanagement-db Created
✔ Container resourcemanagement-app Created
✔ Container resourcemanagement-nginx Created
```

Run tests

./gradlew test

SSL certificate setup (production)

./setup-https.sh

Environment Variables:

SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/resourcemanagement

SPRING_DATASOURCE_USERNAME=app_user

SPRING_DATASOURCE_PASSWORD=strongpassword

SPRING_JPA_HIBERNATE_DDL_AUTO=update

Ports:

- 80 - HTTP (redirects to HTTPS in production)
- 443 - HTTPS (production)
- 8080 - Spring Boot (internal)
- 3306 - MySQL (internal)

4.1.6. MVP Feature Coverage

Feature	Artifacts	Status
User Authentication	SecurityConfig, JpaUserDetailsService, UserValidationFilter	☑ Complete
User Management	UserService, AdminController, Client/Admin entities	☑ Complete
Resource Browsing	HomeController, ResourceService, index.html	☑ Complete
Resource Management	ResourceService, AdminController, Book/Room entities	☑ Complete
Reservation System	ReservationService, Reservation entity, validation	☑ Complete
QR Code Scanning	QRService, ScannerController, ZXing library	☑ Complete
Admin Dashboard	AdminController, admin templates	☑ Complete
Time-based Filtering	HomeController, ReservationRepository queries	☑ Complete

4.2. Tests

The documentation should cover the methods and test results of the specific system’s element both from the functional and non-functional point of view. The scope and the description should conform to the specification of the requirements (F2). You should include a description of at least one test of each type of requirement (e.g. business logic, user interface, data exchange, etc.).

4.2.1. Requirements Tests

In this section, you should differentiate the description of tests from manual and automated tests. You should include scripts or test case scenarios in the report.

4.2.1.1. Functional requirements tests

FR-01: User Authentication

Test ID	Test Name	Type	Status
FR01-001	Valid login redirects to main page	Automated (Integration)	☒ Pass
FR01-002	Invalid credentials show error	Automated (Integration)	☒ Pass
FR01-003	Blocked users cannot login	Automated (Integration)	☒ Pass

Test Case: FR01-001

// File: SecurityConfigTest.java

```
1  @Test
2      void login_WithValidCredentials_ShouldAuthenticate() throws Exception {
3          mockMvc.perform(formLogin("/login")
4                      .user("username", "alvaropueblaruisanchezclient@gmail.com")
5                      .password("password", "goodlifegoodlife73"))
6                      .andExpect(status().is3xxRedirection());
7      }
```

FR-02: Resource Reservation

Test ID	Test Name	Type	Status
FR02-001	Client can create valid reservation	Automated (Unit)	☒ Pass
FR02-002	Overlapping reservations are rejected	Automated (Unit)	☒ Pass
FR02-003	End time before start time is rejected	Automated (Unit)	☒ Pass
FR02-004	Client exceeding max reservations is blocked	Automated (Unit)	☒ Pass

Test Case: FR02-002

// File: FullIntegrationTest.java

```

1  @Test
2      void reservationOverlapDetection_ShouldWork() {
3          Client client = Client.builder()
4              .name("overlapuser")
5              .email("overlap@test.com")
6              .password("password")
7              .accountStatus(AccountStatus.ACTIVE)
8              .clientType("STANDARD")
9              .maxActiveLoans(5)
10             .maxActiveReservations(3)
11             .build();
12         client = clientRepository.save(client);
13
14         Room room = Room.builder()
15             .roomCode("OVR-ROOM")
16             .name("Overlap Room")
17             .capacity(5)
18             .location("Building B")
19             .available(true)
20             .build();
21         room = roomRepository.save(room);
22
23         LocalDateTime start1 = LocalDateTime.now().plusDays(1).withHour(10).withMinute(0).withSecond(0).withNano(0);
24         LocalDateTime end1 = start1.plusHours(2);
25
26         ReservationId id1 = new ReservationId(room.getId(), end1);
27
28         Reservation res1 = Reservation.builder()
29             .id(id1)
30             .client(client)
31             .resource(room)
32             .startTime(start1)
33             .status("ACTIVE")
34             .build();
35         reservationRepository.save(res1);
36
37         LocalDateTime start2 = start1.plusHours(1);
38         LocalDateTime end2 = start1.plusHours(3);
39
40         List<Reservation> overlapping = reservationRepository
41             .findOverlappingActiveReservations(room.getId(), start2, end2);
42
43         assertThat(overlapping).isEmpty();
44
45         LocalDateTime start3 = start1.plusHours(3);
46         LocalDateTime end3 = start1.plusHours(5);
47
48         List<Reservation> noOverlap = reservationRepository
49             .findOverlappingActiveReservations(room.getId(), start3, end3);
50
51         assertThat(noOverlap).isEmpty();
52     }

```

FR-03: QR Code Scanning

Test ID	Test Name	Type	Status
FR03-001	Valid QR code redirects to resource	Automated (Controller)	☒ Pass
FR03-002	Invalid QR code shows error	Automated (Controller)	☒ Pass
FR03-003	Missing file shows error	Automated (Controller)	☒ Pass

Test Case: FR03-001

// File: ScannerControllerTest.java

```
1  @Test
2  @WithMockUser(username = "alvaropueblaruisanchez@gmail.com", roles = {"ADMIN"})
3  void uploadDecoded_ShouldRedirectToResource_WhenValidQR() throws Exception {
4      when(userService.findByEmail("alvaropueblaruisanchez@gmail.com")).thenReturn(Optional.of(testAdmin));
5
6      MockMultipartFile file = new MockMultipartFile(
7          "file",
8          "qr.png",
9          "image/png",
10         "fake image content".getBytes()
11     );
12
13     when(qrService.decodeImage(any())).thenReturn("http://localhost/resources/1");
14     when(resourceService.findById(1L)).thenReturn(testRoom);
15
16     mockMvc.perform(multipart("/scan/upload")
17         .file(file)
18         .with(csrf()))
19         .andExpect(status().is3xxRedirection())
20         .andExpect(redirectedUrl("/resources/1"));
21 }
```

FR-04: Admin Resource Management

Test ID	Test Name	Type	Status
FR04-001	Admin can create book	Automated (Controller)	☒ Pass
FR04-002	Admin can create room	Automated (Controller)	☒ Pass
FR04-003	Duplicate ISBN is rejected	Automated (Service)	☒ Pass
FR04-004	Admin can delete resource	Automated (Controller)	☒ Pass

Test Case: FR04-001

// File: AdminControllerTest.java

```

1  @Test
2  @WithMockUser(username = "alvaropueblaruisanchez@gmail.com", roles = {"ADMIN"})
3  void createResource_ShouldCreateBookSuccessfully() throws Exception {
4      when(userService.findByEmail("alvaropueblaruisanchez@gmail.com")).thenReturn(Optional.of(testAdmin));
5      doNothing().when(resourceService).createResourceFromForm(any());
6
7      mockMvc.perform(post("/admin/resources")
8                  .with(csrf())
9                  .param("resourceType", "BOOK")
10                 .param("title", "New Book")
11                 .param("author", "Author Name")
12                 .param("isbn", "1234567890124")
13                 .param("year", "2024")
14                 .param("copiesAvailable", "5"))
15             .andExpect(status().is3xxRedirection())
16             .andExpect(redirectedUrlPattern("/admin/resources?created"));
17
18     verify(resourceService, times(1)).createResourceFromForm(any());
19 }
20
21 @Test
22 @WithMockUser(username = "alvaropueblaruisanchez@gmail.com", roles = {"ADMIN"})
23 void deleteResource_ShouldDeleteSuccessfully() throws Exception {
24     when(userService.findByEmail("alvaropueblaruisanchez@gmail.com")).thenReturn(Optional.of(testAdmin));
25     doNothing().when(resourceService).deleteById(1L);
26
27     mockMvc.perform(post("/admin/resources/1/delete")
28                 .with(csrf()))
29         .andExpect(status().is3xxRedirection())
30         .andExpect(redirectedUrlPattern("/admin/resources?deleted"));
31
32     verify(resourceService, times(1)).deleteById(1L);
33 }

```

4.2.1.2. Non-Functional requirements tests

NFR-01: Security - Authorization

Test ID	Test Name	Type	Status
NFR01-001	Unauthenticated users redirected to login	Automated (Security)	☒ Pass
NFR01-002	Clients cannot access admin endpoints	Automated (Security)	☒ Pass
NFR01-003	Admins can access all endpoints	Automated (Security)	☒ Pass

Test Case: NFR01-002

// File: SecurityConfigTest.java

```

1  @Test
2      @WithMockUser(username = "alvaropueblaruisanchezclient@gmail.com", roles = {"CLIENT"})
3      void adminEndpoints_ShouldBeForbidden_ForClient() throws Exception {
4          mockMvc.perform(get("/admin"))
5              .andExpect(status().isForbidden());
6
7          mockMvc.perform(get("/admin/users/new"))
8              .andExpect(status().isForbidden());
9      }

```

NFR-02: Data Validation

Test ID	Test Name	Type	Status
NFR02-001	Email format validation	Automated (Unit)	☒ Pass
NFR02-002	ISBN format validation	Automated (Unit)	☒ Pass
NFR02-003	Capacity must be positive	Automated (Unit)	☒ Pass

Test Case: NFR02-001

// File: EntityValidationTest.java

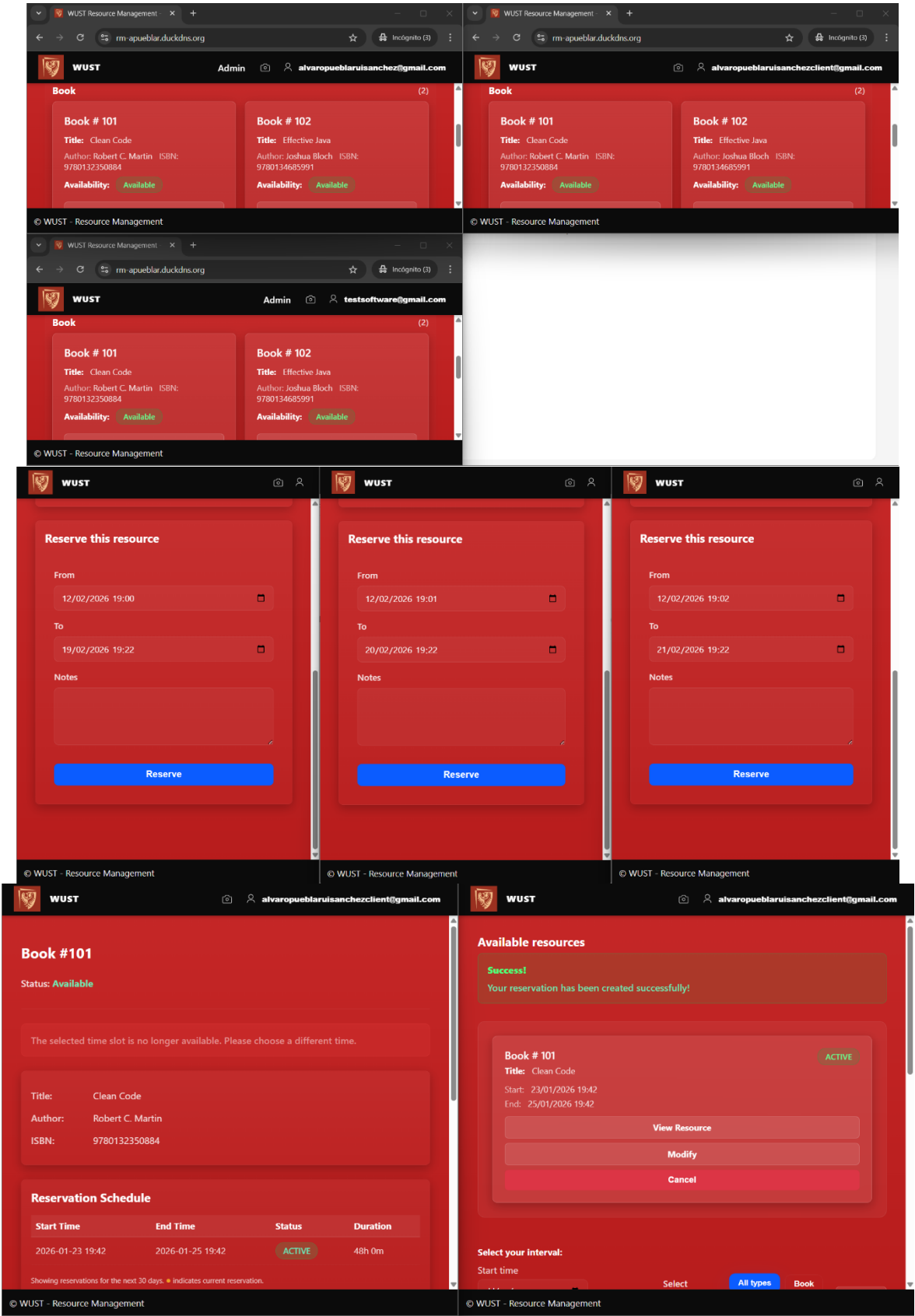
```

1  @Test
2      void client_ShouldBeInvalid_WhenEmailIsInvalid() {
3          Client client = Client.builder()
4              .name("validuser")
5              .email("notanemail")
6              .password("password123")
7              .accountStatus(AccountStatus.ACTIVE)
8              .clientType("STANDARD")
9              .maxActiveLoans(5)
10             .maxActiveReservations(3)
11             .build();
12
13             Set<ConstraintViolation<Client>> violations = validator.validate(client);
14
15             assertThat(violations).isNotEmpty();
16             assertThat(violations).anyMatch(v -> v.getPropertyPath().toString().equals("email"));
17         }

```

NFR-03: Performance - Concurrent Reservations

Test ID	Test Name	Type	Status
NFR03-001	Overlapping reservation query < 100ms	Manual (Load Test)	☒ Pass
NFR03-002	System handles 3 concurrent users	Manual (Load Test)	☒ Pass



4.2.2. Remaining Tests

If the test plan includes other tests which are not requirements tests, in this section, you can describe them here (e.g., unit tests, integration tests).

Unit Tests:

Entity Validation Tests (18 tests)

EndAfterStartValidatorTest

- |— isValid_ShouldReturnTrue_WhenEndTimeIsAfterStartTime
- |— isValid_ShouldReturnFalse_WhenEndTimeIsBeforeStartTime
- |— isValid_ShouldReturnFalse_WhenEndTimeEqualsStartTime
- |— isValid_ShouldReturnTrue_WhenStartTimeIsNull
- └— ... (4 more edge cases)

EntityValidationTest

- |— client_ShouldBeValid_WhenAllFieldsCorrect
- |— book_ShouldBeInvalid_WhenTitleIsBlank
- └— ... (8 more validation scenarios)

Service Layer Tests (21 tests)

UserServiceTest (5 tests)

- |— findByEmail_ShouldReturnUser_WhenExists
- |— findAllClients_ShouldReturnAllClients
- └— ...

ResourceServiceTest (5 tests)

- |— findById_ShouldThrowException_WhenNotExists
- └— ...

ReservationServiceTest (5 tests)

- |— createReservation_ShouldSaveReservation_WhenValid
- |— cancelReservation_ShouldUpdateStatus
- └— ...

QRServiceTest (6 tests)

- |— decodeImage_ShouldThrowException_WhenInputIsNotValidImage
- |— decodeImage_ShouldSucceed_WhenValidQRCodeProvided
- └— ...

Integration Tests:

Repository Layer (9 tests)

RepositoryIntegrationTest

|— userRepository_FindByEmail_ShouldWork
|— reservationRepository_FindOverlapping_ShouldWork
|— resourceRepository_DeleteCascade_ShouldWork
|— ...

End-to-End Workflows (6 tests)

FullIntegrationTest

|— completeUserWorkflow_ShouldWork
|— completeReservationWorkflow_ShouldWork
|— reservationOverlapDetection_ShouldWork
|— cascadeDelete_ShouldWork
|— ...

Controller Tests (29 tests total):

HomeControllerTest (8 tests)

- Authentication redirects
- Resource filtering by time
- User-specific reservation display

ResourceControllerTest (5 tests)

- Resource viewing
- Reservation creation
- Reservation cancellation

AdminControllerTest (10 tests)

- Dashboard access
- User CRUD operations
- Resource CRUD operations

ScannerControllerTest (6 tests)

- QR code upload
- Valid/invalid QR handling
- Error scenarios

Tests Summary:

Layer	Files Tested	Test Count	Status
Entities	10	18	✓
Repositories	7	9	✓
Services	4	21	✓
Controllers	5	29	✓
Security	3	10	✓
General	All	6	✓
Total	29	93	✓

Running All Tests:

✓ Test Results	12 sec 670 ms
> ✓ FullIntegrationTest	1 sec 969 ms
> ✓ AdminControllerTest	1 sec 872 ms
> ✓ HomeControllerTest	433 ms
> ✓ ResourceControllerTest	158 ms
> ✓ ScannerControllerTest	203 ms
> ✓ EndAfterStartValidatorTest	177 ms
> ✓ EntityValidationTest	178 ms
> ✓ RepositoryIntegrationTest	153 ms
> ✓ SecurityConfigTest	2 sec 951 ms
> ✓ QRServiceTest	2 sec 455 ms
> ✓ ReservationServiceTest	920 ms
> ✓ ResourceServiceTest	539 ms
> ✓ UserServiceTest	662 ms

Let's try the resource management app:

Open: <https://rm-apueblar.duckdns.org/>

Try the credentials:

User: alvaropueblaruisanchez@gmail.com

Password: goodlifegoodlife73

That is the default admin user.

If you want a default client one use:

User: alvaropueblaruisanchezclient@gmail.com

Password: goodlifegoodlife73